



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н. Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н. Э. Баумана)

---

ФАКУЛЬТЕТ «Информатика и системы управления»

---

КАФЕДРА «Информационная безопасность»

---

## ОТЧЕТ

по Домашнему заданию № 3

по курсу «Математические основы искусственного интеллекта»

на тему: «Методы одномерного поиска»

Вариант № 6

Студент ИУ8-13М  
(Группа)

\_\_\_\_\_  
(Подпись, дата)

Савватеев А. Э.  
(И. О. Фамилия)

Преподаватель

\_\_\_\_\_  
(Подпись, дата)

Каганов Ю. Т.  
(И. О. Фамилия)

2025 г.

## СОДЕРЖАНИЕ

|                             |          |
|-----------------------------|----------|
| <b>Введение</b>             | <b>3</b> |
| Цель работы . . . . .       | 3        |
| Постановка задачи . . . . . | 3        |
| <b>Практическая часть</b>   | <b>3</b> |
| <b>Заключение</b>           | <b>6</b> |
| <b>ПРИЛОЖЕНИЕ А</b>         | <b>8</b> |

## Введение

### Цель работы

Целью данной работы является изучение методов одномерного поиска, а также применение их для нахождения экстремумов функций нескольких переменных. В ходе работы будут рассмотрены необходимые и достаточные условия экстремума первого и второго порядка.

### Постановка задачи

Требуется найти безусловный минимум функции  $f(x) = 5x^6 - 36x^5 - \frac{165}{2}x^4 - 60x^3 + 36$ ;  $x_0 = -13$  с помощью метода половинного деления, золотого сечения и метода Фибоначчи, а также методом квадратичной и кубической интерполяции

## Практическая часть

В приложении А.1 и А.2 приведен исходный код программы, реализующей методы одномерного поиска. В данной программе реализованы следующие методы: метод половинного деления, метод золотого сечения, метод Фибоначчи, метод квадратичной интерполяции и метод кубической интерполяции.

Начнем анализ работы программы с визуализации функции  $f(x)$  на интервале  $[-30, 30]$ . Для этого воспользуемся библиотекой `matplotlib`. На рисунке 1 представлена графическая интерпретация функции. Так как функция является многочленом высокого порядка, то ее значения на краях интервала могут быть очень большими, поэтому ось  $y$  имеет шкалу, кратную  $1e9$ .

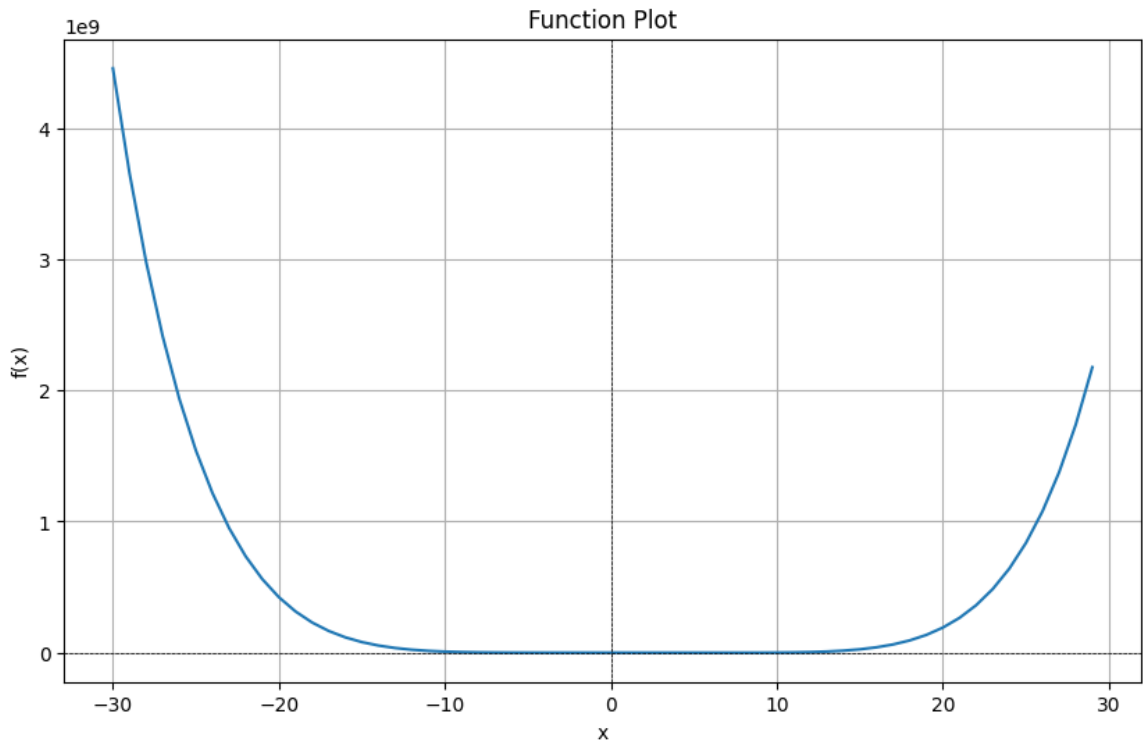


Рисунок 1 – График функции  $f(x)$

Так как весь код приведен в приложении, то здесь мы проанализируем только результаты работы программы. Результатом работы Алгоритма Свенна стал начальный интервал неопределенности  $[-3, 27]$ , что выглядит вполне разумно, так как на этом интервале функция имеет только один минимум. Далее, было решено получить корректный ответ с помощью функции `argmin` из библиотеки `numpy`. Результат работы функции `argmin` показал, что минимум функции  $f(x)$  находится в точке  $x = 7.6$ , что соответствует значению функции  $f(7.6) = -250830$ . Визуализируем окрестности данной точки, чтобы

убедиться в корректности полученного результата. На рисунке 2 представлена графическая интерпретация функции  $f(x)$  на интервале  $[5, 13]$ . Как видно из графика, функция действительно имеет минимум в между 7 и 8.

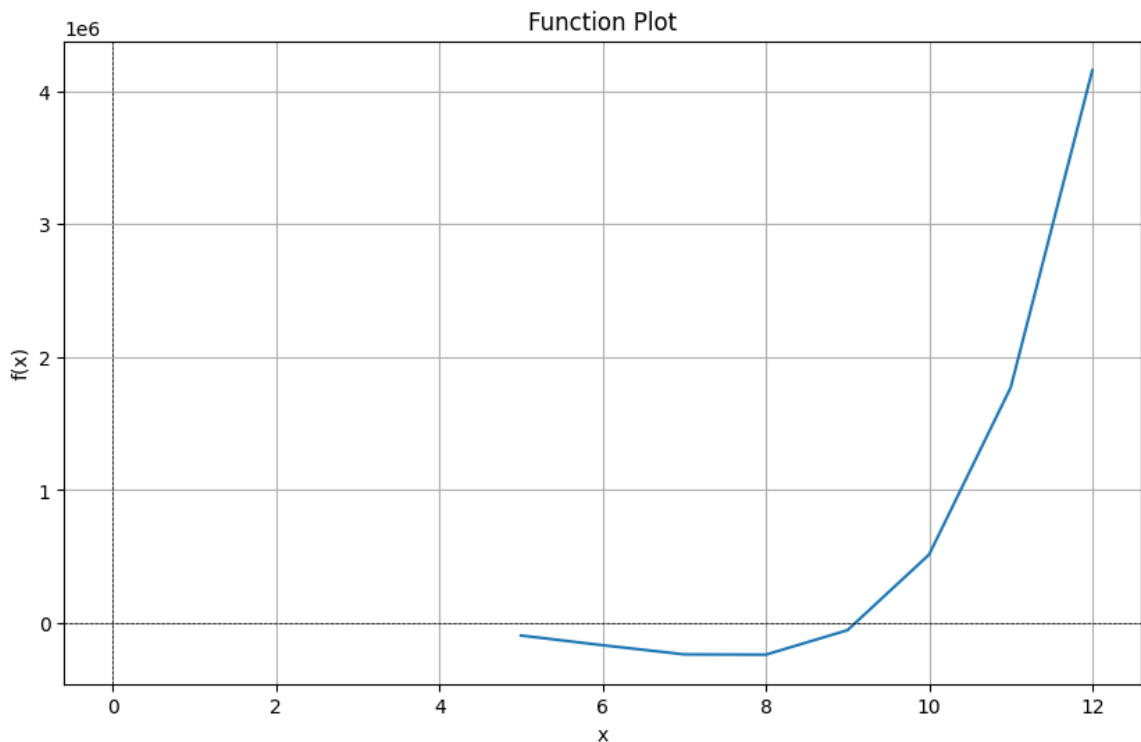


Рисунок 2 – График функции  $f(x)$  на интервале  $[5, 13]$

Теперь приведем результаты работы программы для каждого из методов.

- Метод половинного деления:
  - Начальный интервал неопределенности:  $[-3, 27]$
  - Минимум функции  $f(x)$  находится в точке  $x = 7.56$
- Метод золотого сечения:
  - Начальный интервал неопределенности:  $[-3, 27]$
  - Минимум функции  $f(x)$  находится в точке  $x = 7.56$
- Метод Фибоначчи:
  - Начальный интервал неопределенности:  $[-3, 27]$
  - Минимум функции  $f(x)$  находится в точке  $x = 7.56$
- Метод квадратичной интерполяции:

- Начальный интервал неопределенности:  $[-3, 27]$
  - Минимум функции  $f(x)$  находится в точке  $x = 7.559$
- Метод кубической интерполяции:
  - Начальный интервал неопределенности:  $[-3, 27]$
  - Минимум функции  $f(x)$  находится в точке  $x = 7.56$

## Заключение

В ходе выполнения работы были изучены методы одномерного поиска, а также применены их для нахождения экстремумов функции нескольких переменных. Были рассмотрены необходимые и достаточные условия экстремума первого и второго порядка. В результате работы были получены корректные результаты для всех методов одномерного поиска. Все методы показали одинаковый результат, что подтверждает корректность реализации алгоритмов. Минимум функции  $f(x)$  был найден в точке  $x = 7.56$  с значением функции  $f(7.56) = -250830$ .

## ПРИЛОЖЕНИЕ А

Листинг А.1 – Исходный код программы 3.1

```
1  #%%
2  from matplotlib import pyplot as plt
3  def f(x):
4      return 5 * x**6 - 36 * x**5 - (165 / 2) * x**4 - 60 * x**3 +
        36
5
6  plt.figure(figsize=(10, 6))
7  x_vals = [i for i in range(-30, 30)]
8  y_vals = [f(x) for x in x_vals]
9  plt.plot(x_vals, y_vals)
10 plt.title("Function Plot")
11 plt.xlabel("x")
12 plt.ylabel("f(x)")
13 plt.grid()
14 plt.axhline(0, color='black', linewidth=0.5, ls='--')
15 plt.axvline(0, color='black', linewidth=0.5, ls='--')
16 plt.show()
17 #%%
18 # рассчитать минимум функции на интервале [-30, 30] с шагом 0.1
19 import numpy as np
20 x = np.arange(-30, 30, 0.1)
21 y = f(x)
22 min_index = np.argmin(y)
23 min_x = x[min_index]
24 min_y = y[min_index]
25 print(f"Minimum found at x = {min_x}, f(x) = {min_y}")
26 #%%
27 def find_interval():
28     x = [-13]
29     x0 = x[0]
30     t = 5
31     a, b, delta, k = 0, 0, 0, 0
32
33     if f(x0 - t) >= f(x0) <= f(x0 + t):
34         a = x0 - t
35         b = x0 + t
36         return a, b
```

```

37
38     if f(x0 - t) >= f(x0) >= f(x0 + t):
39         delta = t
40         a = x0
41         x.append(x0 + delta)
42     elif f(x0 - t) <= f(x0) <= f(x0 + t):
43         delta = -t
44         b = x0
45         x.append(x0 + delta)
46
47     while f(x[k + 1]) < f(x[k]):
48         k += 1
49         x.append(x0 + 2 ** k * delta)
50         if x[k + 1] < f(x[k]) and delta == t:
51             a = x[k]
52         elif x[k + 1] < f(x[k]) and delta == -t:
53             b = x[k]
54     if delta == t:
55         b = x[-1]
56     elif delta == -t:
57         a = x[-1]
58     return a, b
59
60 a, b = find_interval()
61 print("Interval found: [{a}, {b}]")
62 #%%
63 # Алгоритм стягивающихся отрезков
64 def contracting_intervals(a_left, b_right, epsilon=1e-5):
65     k = 0
66     x_mid = (a_left + b_right) / 2
67     while abs(b_right - a_left) > epsilon:
68         l = abs(b_right - a_left)
69         k += 1
70         y = a_left + l / 4
71         z = b_right - l / 4
72         if f(y) < f(x_mid):
73             b_right = x_mid
74             a_left = a_left
75             x_mid = y
76         else:
77             if f(z) < f(x_mid):

```

```

78         a_left = x_mid
79         b_right = b_right
80         x_mid = z
81     else:
82         a_left = y
83         b_right = z
84
85     return x_mid, a_left, b_right
86
87 x_min, a_final, b_final = contracting_intervals(a, b)
88 print(f"Minimum found at x = {x_min} within interval [{a_final},
89       {b_final}]")
89 #%%
90 # Метод золотого сечения
91 def golden_section(a_left, b_right, epsilon=1e-5):
92     k = 0
93     y = a_left + (3 - 5 ** 0.5) / 2 * (b_right - a_left)
94     z = a_left + b_right - y
95     while abs(b_right - a_left) > epsilon:
96         k += 1
97         if f(y) <= f(z):
98             b_right = z
99             z = y
100             y = a_left + b_right - y
101         else:
102             a_left = y
103             y = z
104             z = a_left + b_right - z
105
106     return (a_left + b_right) / 2, a_left, b_right
107
108 x_min_gs, a_final_gs, b_final_gs = golden_section(a, b)
109 print(f"Minimum found at x = {x_min_gs} within interval
110       [{a_final_gs}, {b_final_gs}]")
110 #%%
111 # Метод Фибоначчи
112 def fibonacci_method(a_left, b_right, epsilon=1e-5):
113     F = [1, 1]
114     while F[-1] < (b_right - a_left) / epsilon:
115         F.append(F[-1] + F[-2])
116

```

```

117     y = a_left + F[-3] / F[-1] * (b_right - a_left)
118     z = a_left + F[-2] / F[-1] * (b_right - a_left)
119     N = len(F) - 1
120     for k in range(N - 3):
121         if f(y) <= f(z):
122             b_right = z
123             z = y
124             y = a_left + F[N-k-3] / F[N-k-1] * (b_right - a_left)
125         else:
126             a_left = y
127             y = z
128             z = a_left + F[N-k-2] / F[N-k-1] * (b_right - a_left)
129
130     y = (a_left + b_right) / 2
131     z = y + epsilon
132     if f(y) <= f(z):
133         b_right = z
134     else:
135         a_left = y
136
137     return (a_left + b_right) / 2, a_left, b_right
138
139 x_min_fib, a_final_fib, b_final_fib = fibonacci_method(a, b)
140 print(f"Minimum found at x = {x_min_fib} within interval
      [{a_final_fib}, {b_final_fib}]")

```

### Листинг A.2 – Исходный код программы 3.2

```

1 def quadratic_interpolation_by_algorithm(func, a1_start, da,
2     eps=1e-6, delta=1e-6, max_iter=100):
3
4     a1 = a1_start
5
6     def initialize_points(current_a1):
7         local_a2 = current_a1 + da
8
9         local_phi1 = func(current_a1)
10        local_phi2 = func(local_a2)
11
12        if local_phi1 > local_phi2:
13            local_a3 = current_a1 + 2 * da
14        else:
15            local_a3 = current_a1 - 2 * da

```

```

15         local_phi3 = func(local_a3)
16
17         return current_a1, local_a2, local_a3, local_phi1,
18             local_phi2, local_phi3
19
20 a1, a2, a3, phi1, phi2, phi3 = initialize_points(a1)
21
22 iterations = 0
23 while iterations < max_iter:
24     iterations += 1
25
26     if phi1 <= phi2 and phi1 <= phi3:
27         a_min, phi_min = a1, phi1
28     elif phi2 <= phi1 and phi2 <= phi3:
29         a_min, phi_min = a2, phi2
30     else:
31         a_min, phi_min = a3, phi3
32
33     numerator = (a2**2 - a3**2) * phi1 + (a3**2 - a1**2) *
34         phi2 + (a1**2 - a2**2) * phi3
35     denominator = 2 * ((a2 - a3) * phi1 + (a3 - a1) * phi2 +
36         (a1 - a2) * phi3)
37
38     # Проверка знаменателя на ноль (согласно примечанию в
39     # III.7)
40
41     if abs(denominator) < 1e-12:
42         print(f"Итерация {iterations}: Знаменатель равен
43             нулю. Перезапуск с = {a_min:.4f}")
44         a1, a2, a3, phi1, phi2, phi3 =
45             initialize_points(a_min)
46         continue # Переходим к следующей итерации (начинаем
47             с III.6)
48
49     a_bar = numerator / denominator
50     phi_bar = func(a_bar)
51
52     if abs(a_bar) > 1e-9 and abs(phi_bar) > 1e-9:
53         cond1 = abs((phi_min - phi_bar) / phi_bar) < eps
54         cond2 = abs((a_min - a_bar) / a_bar) < delta
55         # а) если оба условия выполнены
56         if cond1 and cond2:

```

```

49         print(f"Процедура успешно завершена на итерации
              {iterations}.")
50         return a_bar
51
52     x_min_bound = min(a1, a2, a3)
53     x_max_bound = max(a1, a2, a3)
54
55     if not (x_min_bound <= a_bar <= x_max_bound):
56         print(f"Итерация {iterations}: _bar ({a_bar:.4f})
              вне интервала. Перезапуск с _bar")
57         a1, a2, a3, phi1, phi2, phi3 =
              initialize_points(a_bar)
58         continue
59
60     else:
61         all_points = sorted([(a1, phi1), (a2, phi2), (a3,
              phi3), (a_bar, phi_bar)], key=lambda p: p[0])
62
63         best_point = min(all_points, key=lambda p: p[1])
64         best_idx = all_points.index(best_point)
65
66         if best_idx == 0:
67             new_points = all_points[0:3]
68         elif best_idx == len(all_points) - 1:
69             new_points = all_points[-3:]
70         else:
71             new_points = all_points[best_idx - 1 : best_idx
              + 2]
72
73         (a1, phi1), (a2, phi2), (a3, phi3) = [(p[0], p[1])
              for p in new_points]
74
75     print("Достигнуто максимальное количество итераций.")
76     return a_min
77
78 def f(x):
79     return 5 * x**6 - 36 * x**5 - (165 / 2) * x**4 - 60 * x**3 +
              36
80
81 start_point = -10
82 step = 3

```

```

83 minimum_x = quadratic_interpolation_by_algorithm(f, start_point,
    step)
84
85 print("\n--- Результат ---")
86 print(f"Найденный минимум в точке x = {minimum_x:.6f}")
87 #%%
88 def f(x):
89     return 5 * x**6 - 36 * x**5 - (165 / 2) * x**4 - 60 * x**3 +
        36
90 def df(x):
91     return 30 * x**5 - 180 * x**4 - 330 * x**3 - 180 * x**2
92
93 def cubic_interpolation_literal(func, grad, a0, da, eps=1e-6,
    delta=1e-6, max_iter=100):
94     g0 = grad(a0)
95
96     ak = a0
97     k = 0
98
99     while True:
100         gak = grad(ak)
101
102         if g0 < 0:
103             ak_next = ak + (2**k) * da
104         else:
105             ak_next = ak - (2**k) * da
106
107         gak_next = grad(ak_next)
108
109         if gak * gak_next <= 0:
110             a_M_minus_1 = ak
111             a_M = ak_next
112             break
113
114         ak = ak_next
115         k += 1
116         if k > 50:
117             raise RuntimeError("Не удалось найти интервал,
                содержащий минимум.")
118
119     a1 = a_M_minus_1

```

```

120     a2 = a_M
121     print(f"Интервал найден:  = {a1:.4f},  = {a2:.4f}")
122
123     iterations = 0
124     while iterations < max_iter:
125         iterations += 1
126
127         phi1, phi1_prime = func(a1), grad(a1)
128         phi2, phi2_prime = func(a2), grad(a2)
129
130         z = (3 * (phi1 - phi2) / (a2 - a1)) + phi1_prime +
            phi2_prime
131
132         w_term = z**2 - phi1_prime * phi2_prime
133         if w_term < 0:
134             w_term = 0 #
135
136         if a1 < a2:
137             w = math.sqrt(w_term)
138         else:
139             w = -math.sqrt(w_term)
140
141         mu_denominator = (phi2_prime - phi1_prime + 2 * w)
142         if abs(mu_denominator) < 1e-12:
143             a_bar = a1 + 0.5 * (a2 - a1)
144         else:
145             mu = (phi2_prime + w - z) / mu_denominator
146             if mu < 0: a_bar = a2
147             elif mu > 1: a_bar = a1
148             else: a_bar = a2 - mu * (a2 - a1)
149
150         phi_bar = func(a_bar)
151
152         if phi_bar > phi1:
153             while phi_bar > phi1 and iterations < max_iter:
154                 a_bar = a_bar - 0.5 * (a_bar - a1)
155                 phi_bar = func(a_bar)
156                 iterations += 1
157
158         phi_bar_prime = grad(a_bar)
159         cond1 = abs(phi_bar_prime) <= eps

```

```

160         cond2 = False
161     if abs(a_bar) > 1e-9:
162         cond2 = abs(a_bar - a1) / abs(a_bar) <= delta
163
164     if cond1 and cond2:
165         print(f"\nАлгоритм сошелся за {iterations}
166             итераций.")
167         return a_bar
168
169     g_bar = phi_bar_prime
170     g1 = phi1_prime
171
172     if g_bar * g1 < 0:
173         a2 = a1
174         a1 = a_bar
175     else:
176         a1 = a_bar
177
178     print("\nДостигнуто максимальное количество итераций.")
179     return a_bar
180
181 start_point = -13
182 step = 1
183 minimum_x = cubic_interpolation_literal(f, df, start_point, step)
184
185 print("\n--- Результат ---")
186 print(f"Найденный минимум в точке x = {minimum_x:.6f}")
187 print(f"Значение функции в этой точке = {f(minimum_x):.6f}")
188 print(f"Значение производной в этой точке = {df(minimum_x):.6f}")

```